

Build the trial log before the dashboard

Deep research · L/S Gamma Desk, QUANTT

2026-10-01

Question: Should the L/S Gamma desk build a dashboard that tracks backtesting and research, separate from IBKR live paper trading?

Method: Deep research: three parallel web research tracks (tooling, research-tracking practice, desk fit) synthesized into a report. Notes for each track are filed alongside.

Yes, the L/S Gamma desk should build backtest and research tracking that is separate from the IBKR paper-trading system. It should not start with an interactive dashboard, though. The research makes one component essential: an **automatic, append-only trial registry** that records every backtest configuration and its daily return series. Every overfitting control the desk already plans to use depends on it. The Deflated Sharpe Ratio needs the number of trials and the variance of their Sharpes, and PBO needs the full matrix of trial returns. QuantConnect’s own research gauge rates projects with more than 70 backtests as “Probably Overfitting.” The right order follows from that. First freeze the output schema. Then have the backtester write a static per-run report and a registry row. A small local Streamlit or MLflow comparison view comes last, once there are dozens of runs. Everything should stay on club-controlled infrastructure, because R2/WRDS data is licensed to Queen’s. The report layer itself takes days. The real bottleneck is the option-data layer it depends on (R2 has option trades but no quotes and no intraday SPY), which will take weeks. The dashboard question is therefore mostly about the order of work. Within that order, the most important display is not the equity curve. It is the trial counter and deflated Sharpe placed next to every raw Sharpe.

1 Every overfitting correction needs a trial count the desk does not yet record

The case for research tracking does not rest on convenience. Each standard correction for backtest overfitting takes the record of what was tried as a required input. Bailey and López de Prado’s Deflated Sharpe Ratio adjusts a Sharpe for skew, kurtosis, track-record length, **“the variance of the SRs tested,”** and **“the number of independent trials... (N)”** (DSR slides). Their worked example is sobering: an annualised Sharpe of 2.5 found after 100 trials, with skew -3 , is not a discovery at 95%. That skew is typical of short-vol payoffs. López de Prado writes that without the trial count “it is not possible to determine the Family-Wise Error Rate... False Discovery Rate... Probability of Backtest Overfitting (PBO) or similar,” and adds that about **20 iterations of “research through backtesting” are enough to produce a false 5% discovery** (López de Prado 2018). Arnott, Harvey and Markowitz make “Keep Track of What Is Tried” the first rule of their multiple-testing protocol. They extend it to variable combinations and data-transformation choices, such as which volatility scaling was used (Arnott et al. 2019).

The empirical evidence points the same way. Across 888 Quantopian algorithms, backtest Sharpe explained **less than 2.5% of the variance in out-of-sample Sharpe**, and “the more backtesting a quant has done for a strategy, the larger the discrepancy” (CXO summary of Wiecki et al.). Bailey et al. show that with five years of daily data, trying 45 or more independent variations makes a Sharpe of at least 1.0 by pure chance “more than likely” (Bailey et al.). For this desk, every VRP band, forecaster (GARCH or HAR), hedge-frequency setting, exit rule and LG/SG structure tried counts toward N. Two design requirements follow. First, logging must be **automatic**, because a manual log undercounts trials, and the trial count is exactly the number that must not be undercounted. Second, the log must keep each trial’s **return series**, not just its summary statistics, because $V[SR]$, the correlation-based “effective N” (López de Prado & Lewis 2019) and the PBO matrix (Bailey et al., PBO) all need it. The repo’s own walk-forward report

already asks the desk to “log every configuration tried” and to report a DSR across all trials (`research/strategy_research/2026-10-01-walk-forward-backtest-without-look-ahead.tex`). Nothing in `src/` does this today, and `src/lsgamma/backtest/` holds only a `.gitkeep`.

Firm practice treats this as process governance, not just tooling. Man AHL tracks “how many strategies are proposed originally, and how far they make it through the process,” counts how often researchers access the data, and after its PBO work required researchers to “write down the data partitioning and the methodology and expectations” before testing (Man Group). For a PM supervising four analysts, Arnott et al.’s warning is directly relevant: research assistants “have an incentive to please their supervisor by presenting results that support the supervisor’s hypothesis” (Arnott et al. 2019). A registry that lists killed trials next to promoted ones is the structural answer to that incentive.

2 Separation protects the only true out-of-sample evidence

Keeping research tracking apart from the live paper system is well supported, although no source argues explicitly for “separate dashboards.” The case is inferred from research on hold-out contamination. Arnott et al. state that “the only true out of sample is the live trading experience,” and that modifying a model after it fails out of sample “is no longer an out-of-sample test. It is overfitting” (Arnott et al. 2019). Dwork et al. show that adaptively reusing a hold-out “can easily lead to overfitting to the holdout set itself” (Science 2015). Blum and Hardt document the same thing on Kaggle, where leaderboard scores were inflated relative to final scores (The Ladder). A leaderboard is simply a dashboard over a hold-out. A combined view that shows twelve years of backtest next to a few months of paper P&L invites exactly this kind of loop: someone tunes the VRP band because paper trading looked weak. That would spend both the 2025-07 to 2026-07 hold-out and the paper record.

The two systems also serve different audiences and different data. Research views compare many mostly-failed trials over licensed R2/WRDS history. Paper monitoring tracks one deployed specification over free yfinance and IBKR data. The practical rule is to tag every row with `source` $\in$ `{backtest, paper}` and a `run_id`, keep the data in separate directories (`results/paper/` and `results/backtest/<run_id>/`), and join them only in the report layer. Paper results should be treated as a one-shot test of a pre-registered spec version. Any change to the strategy restarts the clock and counts as a new trial.

Comparing the two still matters, and the right method is QuantConnect-style reconciliation. Replay the backtest over exactly the paper dates from the same starting position, then attribute each gap to data, execution, timing or logic. QuantConnect describes perfect reconciliation as “an exact overlap between its live and OOS backtest equity curves” and lists the usual causes of drift: data providers, discrete versus continuous data, fee and slippage models, fill latency, and pre-existing positions (QuantConnect). The desk already knows several of these will apply. Delta comes from IBKR `modelGreeks` live but would come from in-house Black-Scholes in the backtest. ATM IV comes from the IBKR chain live but from R2 trades in the backtest. Live limit orders at mid + \$0.05 can miss, and live close-row P&L excludes the hedge unwind (`src/lsgamma/trading/`, the desk’s progress log). Operational statistics such as exit-reason mix, days held, hedge trades per position and cost per trade will converge much faster than Sharpe on a few dozen paper trades. They are the right things to compare first.

3 Contents: P&L attribution and a deflated Sharpe beat a prettier equity curve

Generic tear-sheet metrics are necessary but miss the bet the desk is actually making. For a delta-hedged option, cumulative P&L is approximately $\frac{1}{2} \int \Gamma S^2 (\sigma^2_{\text{realized}} - \sigma^2_{\text{implied}}) dt$: the desk is long realised and short implied variance, sized by dollar gamma (Moontower). Bakshi and Kapadia made delta-hedged gains the standard measure of the volatility risk premium (RFS 2003). The practitioner daily decomposition splits P&L into delta ($\Delta \cdot \Delta S$), gamma ($\frac{1}{2} \Gamma \cdot \Delta S^2$), theta ($\Theta \cdot dt$) and vega ($\nu \cdot \Delta \sigma$), plus a residual from vanna, volga and charm that stays small when moves are moderate (Moontower decomposition). This attribution doubles as the desk’s main debugging tool. A growing residual signals bad marks, a mis-implied spot or a Greeks bug. That matters because historical spot will have to be implied from put-call parity on

R2 trades.

Risk metrics need to respect negative skew. Over 1986 to 2018 the Cboe PUT index, the natural public benchmark for a short-put leg, had monthly **skew -2.09 , kurtosis 12.58 , max drawdown -32.7% and a 40-month longest drawdown** next to a modest Sharpe of 0.65. Bondarenko reports a Stutzer index alongside it because that index “does not assume” normality (Bondarenko 2019, Cboe). The same paper shows VIX averaging 19.3 against 15.1 realised volatility from 1990 to 2018, with a negative annual gap only in 2008. That exhibit is the template for the desk’s vol panel. Library defaults can quietly diverge from the desk’s own definitions. QuantStats, for example, uses parametric-normal VaR and a Sortino denominator taken over all observations (quantstats/stats.py). Metric definitions should therefore be pinned in one tested module.

Priority	View	Why it matters for this desk	Buildable from existing code?
1	Trial counter, effective N, DSR/PSR beside every Sharpe; proposed $\rightarrow$ killed $\rightarrow$ promoted funnel	Makes logging a visible penalty rather than a vanity metric	No: registry + DSR function needed
2	Per-trade “card”: strike, expiry, DTE, entry IV, RV forecast, spread, realised vol over hold, option/hedge/cost/net P&L, exit reason	Exit reasons (time/stop/profit/flip/flat) already exist in trading/exits.py	Mostly: reuse ledger schema, add position id and hedge-unwind P&L
3	Daily Greeks attribution (gamma, theta, vega, delta residual, costs, unexplained)	Tests the actual bet and catches data bugs	No: no Black-Scholes pricer or IV solver exists in src/
4	Vol panel: IV vs forecast vs subsequent realised vol, shaded by signal state; scatter of entry spread vs trade P&L	Cleanest test of whether the VRP signal works	Half: signals.parquet holds forecast/IV/spread; realised-over-hold is new
5	Equity and underwater curve vs SPY and the README’s VIX short-term futures benchmark (and optionally Cboe PUT)	README targets: Sharpe > 1.2 , Sortino > 1.5 , max DD $< 15\%$	No daily NAV series exists yet
6	Skew-aware risk table: skew, kurtosis, historical CVaR, worst day/month, longest drawdown, beta to SPY, correlation to Δ VIX	Sharpe alone flatters short-vol books	Pandas once NAV exists
7	Regime breakdown (VIX percentile, VIX/VIX3M term structure, 200-day trend, long vs short gamma, year) with trade counts	Features already computed in pipeline/features.py	Join only; VIX must be lagged one day
8	Cost drag: gross vs net, net Sharpe vs slippage assumption	R2 has no quotes, so spreads are modelled	New cost model
9	Walk-forward folds, stitched OOS curve, hold-out shown once with a reveal stamp	Protects the 2025-07..2026-07 hold-out	New splitter + guard

Two caveats apply to the table. First, the walk-forward report estimates **at most about 130 non-overlapping trades over twelve years**, so per-regime and per-exit-reason cells will have wide confidence intervals. Every cell should show its trade count and a bootstrap interval, and thin cells should be greyed out. The 10–15-trade cutoff for greying out is judgment, not sourced. Second, the README target “VRP Capture $> 60\%$ of theoretical VRP” has no definition anywhere in the repo. One candidate is net P&L divided by the frictionless gamma-plus-theta term over the hold, but the PM has to define it before any dashboard displays it. Benchmark choice beyond the two README benchmarks, the greying-out cutoffs and slippage grids are strategy decisions for the PM, not defaults.

The evidence also warns against false rigour. Gençay’s 2026 study found that DSR-style deflation alone can miss deliberately biased strategies, and argues for structural controls such as leakage-safe feature registries alongside the statistics (arXiv 2608.27734). Harvey recommends registered reports, meaning the rationale and method are fixed before the data is touched (Harvey 2017). In practice, each registry row should link to a committed spec file and hypothesis written before the run. A polished equity-curve page that shows no N, no cost model and no hold-out discipline is worse than a CSV, because it signals validation that never happened.

4 Tooling: parquet registry and static reports now, MLflow or Streamlit later

The 2025–2026 consolidation of experiment trackers decides much of the tooling question. **Neptune is gone.** OpenAI acquired it, and the service shut down on 2026-03-05, deleting all remaining hosted data (Neptune transition hub; Techsoma). **W&B is now part of CoreWeave Forge**, and sign-in moved to Forge on 2026-09-30 (CoreWeave). Its Academic plan is free for up to 100 seats, but only Enterprise offers bring-your-own-bucket storage (Forge pricing). **DVC moved to lakeFS**, and the future of DVC Studio is unaddressed (DVC FAQ). **Aim** has not had a formal release since May 2025, and a production user’s question about Python 3.13+ support had no maintainer reply (Aim #3414). **MLflow** is the stable option. It is Apache-2.0, released v3.15.0 on 2026-07-31 with a Python 3.14 fix (MLflow releases), and now uses SQLite as its default backend, so `mlflow ui` on a local file is effectively a run registry with a comparison UI (MLflow server docs). Its multi-user basic auth is still marked experimental and ships with a default `admin/password1234` account that must be changed (MLflow auth).

Licensing narrows the choice further. WRDS restricts use to “academic and non-commercial research purposes” (WRDS ToU), discourages sharing raw data outside the institution (WRDS guide), and asks that data be stored “in a secure, private location” and deleted when affiliation ends (WRDS policy). Anything row-level derived from R2 or WRDS, such as option marks, parity-implied spot or per-trade P&L tied to licensed prices, should therefore stay on club-controlled storage, not on W&B or Streamlit Community Cloud. GitHub Pages is ruled out: sites are **public even from a private repo** unless the organisation is on GitHub Enterprise Cloud (GitHub docs). Streamlit Community Cloud allows only one private app per user, and that app would need network access to the results (Streamlit docs). Grafana is built for time-series observability, not run comparison, and its free cloud tier caps active viewers at three (Grafana plans).

Layer	Recommendation	Rationale	Caveat
Run output	<code>results/backtest/<run_id>/</code> with <code>run.json</code> (config, config hash, git SHA, data range, fold, seed) plus the live ledger schema (<code>signals</code> , <code>trades</code> , <code>hedges</code>) and a new <code>daily</code> table (spot, mark, IV, Greeks, hedge shares, NAV, costs)	One report function can read both paper and backtest output	Requires making <code>ledger.ROOT</code> a parameter
Registry	Append-only <code>results/backtest/registry.parquet</code> , written automatically, with a <code>holdout_viewed</code> flag and a path to each run's return series	Supplies <code>N</code> , <code>V[SR]</code> , effective <code>N</code> and <code>PBO</code>	Gitignored; never committed
Metrics	Own <code>metrics.py</code> in pandas, pinned definitions, tested against hand-computed values	Avoids silent library defaults and dependency rot	—
Per-run report	Static HTML/PDF with matplotlib; QuantStats optional	QuantStats 0.0.86 (2026-09-27) makes one-call tearsheets (PyPI)	Its classifiers stop at Python 3.13, so test it on the repo's 3.14/pandas 3 stack or skip it
Comparison UI (later)	Local Streamlit app over the registry, or self-hosted MLflow on SQLite with artifacts in a private R2 prefix	Both free, Python-native, keep data on club infrastructure	Defer until runs number in the dozens
Avoid	Neptune, Aim, GitHub Pages, SaaS hosting of row-level licensed data	Shut down, stalled, public, or licence risk	—

5 Sequence: contract first, dashboard last

The timing follows from where the effort lies. The interactive layer is cheap. The data and backtest layers underneath it are expensive and not yet built. The following estimates are judgment, not sourced, and assume one or two analysts working part-time. Freezing the output contract and parameterising the ledger root takes about a week. A Black-Scholes pricer, IV solver and Greeks with tests take one to two weeks. Turning R2 trades into daily option marks, parity-implied spot and daily ATM IV takes **three to six weeks, the widest error bar**, because nobody has checked how dense SPY ATM trades are around 10:15 or the close. The backtest loop reusing `SIGNALS`, `FORECASTERS`, `selection`, `exits` and `pnl` with a simulated broker takes two to three weeks. The static report takes one to two weeks, as does the registry with DSR, embargoed walk-forward splitter and hold-out guard. An interactive dashboard would add one to three more.

The recommended order is as follows. Fix the output schema **before** writing the backtester, because it determines what any report can show. Have the MVP backtester write a static report and a registry row from its **first** run, because trials that are not logged cannot be counted later. Add walk-forward fold pages and a hold-out page that is written once and flagged in the registry. Build an interactive comparison view **only after** the registry holds dozens of runs and the schemas have stopped changing. Building the dashboard first would mean designing views around data that does not exist yet. It would also lower the cost of each additional look, and the extra looks are what the trial count is meant to penalise. Maintenance risks to plan for now: schema drift between the live and backtest writers (fix with one shared schema module and a pytest covering both), drift in metric definitions, rot in third-party libraries on Python 3.14, contamination of the hold-out, and loss of knowledge through yearly analyst turnover (fix with a single documented report

command).

Several gaps remain open. No source compares static and interactive research dashboards for small teams, and none documents how university trading clubs track research, so the sequencing is reasoned rather than proven. The density of R2 option trades, the licence terms of the R2 tick data’s original vendor, and whether historical Cboe PUT and VIX-futures index series are free to use were not verified. Whether aggregate-only reports may be shared or committed is a decision for the PM after checking with Queen’s library or WRDS.

6 Conclusion

The evidence points to a change in what “dashboard” means for this desk. Its main job is not to show performance. Its job is to keep an honest count of how many times the desk has looked. The output with the strongest evidence behind it is a trial counter with a deflated Sharpe beside every raw Sharpe, a direct remedy for the Quantopian finding that more backtesting predicts worse out-of-sample results. Equity curves are secondary. Separating research tracking from paper trading matters for the same reason: it protects the desk’s two scarce out-of-sample assets, the 2025–2026 hold-out and the paper record, from being tuned against.

The practical consequence is that the “dashboard” work starts as a schema decision this week, not a UI project next term. If the backtester emits the same ledger schema as the live system and writes a registry row on every run, the static reports, DSR, PBO, paper-versus-backtest reconciliation and any later Streamlit or MLflow view all become cheap additions. If it does not, no later dashboard can recover the uncounted trials. Before any view goes live, the PM has four decisions to make: how to define “VRP capture,” which benchmarks to show, whether aggregate reports may be shared, and when the hold-out may be revealed.